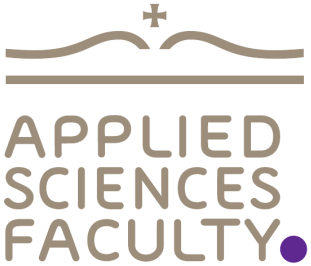
C++ — лекція 18. Виключення





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

"Класичні" способи повідомлення про помилки

Коли під час роботи сталася помилка, функція, котра її викликала, може:

- Завершити виконання програми. Жорстокий і часто неприйнятний метод. Детальніше тут його не обговорюватимемо — коли його достатньо, теми для обговорення немає, але зазвичай можна придумати щось краще. Крім того, бібліотека, яка так робить, зазвичай перевищує свої повноваження, приймаючи рішення за цілу програму.
- Повернути код завершення. Традиційно — нуль, якщо все пройшло успішно, відмінне від нуля значення, якщо виникли проблеми. Однак, якщо функція повертає вказівник, все навпаки — нуль відповідає помилці, відмінне від нуля значення — коректному вказівнику, з яким можна працювати.
- Повернути те чи інше, значення, але змінити глобальний стан програми, наприклад — записати код помилки у глобальну змінну, типу errno.
- Викликати функцію-обробник.

Недоліки кодів помилок

Хоча повертати код помилки — дуже поширений підхід, він має ряд суттєвих недоліків:

```
int result = somefunction();
if( result != 0){
  // Обробка помилки
}
```

- Код помилки слід перевіряти після кожного виклику. Це достатньо нудно і займає багато часу. Природно, програмісти мають велику спокусу пропускати перевірки, що веде до поширення необроблених помилкових ситуацій.
- Навіть якщо програміст дуже старанний, код обробки помилок знаходиться поміж коду, що реалізовує нормальну логіку роботи програми. Це незручно, та й, з точки зору краси та елегантності програми, неправильно.
- Іноді можливості повернути значення просто немає — наприклад, як має сигналізувати про помилку конструктор?
- Використання коду помилок іноді змушує змінювати інтерфейс функцій.

Глобальний стан, типу errno

- Володіє схожими, до кодів завершення, недоліками.
- Додатково може бути проблемою в багатопоточному середовищі.
- Використання, на прикладі errno:

```
errno = 0;
result = sqrt (x);
if (errno != 0){
    ..............
}
```

Перед викликом, що може змінювати errno, слід зануляти її. В сучасних компіляторах errno є потокобезпечним — кожен потік має свою копію.

Виклик обробника у випадку виникнення помилки

- ```
  void library_function (){
      .........
          if (have_error)
              error_handler ();
  }
  ```

  Важливе питання — а що цей обробник може зробити, щоб вирішити проблему? І якщо щось може зробити (локально), не турбуючи того, хто викликав функцію, то які проблеми, хіба це помилка?
  ```

Exceptions — виключення

- C++ пропонує альтернативний метод обробки помилок — виключення.
- Вони призначені обробляти нештатні ситуації, які не можна вирішити локально.
- Нештатна ситуація — це коли викликана функція не може здійснити свою роботу із тих чи інших причин.
- При чому, мається на увазі, що це таки має бути нештатна ситуація — не слід використовувати їх в процесі нормального потоку виконання, (скажімо, не використовуйте для діагностики порожньої стрічки-аргументу, тоді коли порожня стрічка не є аварійною ситуацією). Це може зіпсувати структуру програми і сильно сповільнити її роботу.
- Але ця нештатна ситуація не обов'язково означає катастрофу — навіть в програмі, що успішно працює, цілком може бути буде певна кількість виключень.

Використання виключень

- У місці, де виявлено помилку, генерується **виключення** — створюється об'єкт певного класу, який міститиме інформацію про помилку, і починається послідовний вихід із всіх функцій, із викликом деструкторів всіх локальних змінних[1]. Все рівно, код в місці виникнення помилки, зазвичай не знає, що з нею робити далі.
- Коди вищого рівня, який нарешті знатиме, як повестися, може **перехопити** виключення, "зловити" згаданий вище об'єкт-виключення, з інформацією про помилку, проаналізувати його вміст та здійснити відповідні дії[2].
- Яка помилка сталася, визначається класом об'єкту, а особливості конкретної помилки — описуються його вмістом.
- Важливо, що різні виключення можна перехоплювати окремо, залишаючи обробку помилок тому коду, який компетентний — знатиме, що з нею робити.

---

[1] Цей процес називається розкручуванням стеку (stack unwinding), і служить для того, щоб у випадку помилки коректно "почиститися", звільнити всю виділену пам'ять, позакривати файли, і т.д.

[2] Наприклад, спробувати під'єднатися до сервера ще раз, запропонувати замінити диск, попередити користувача і завершити роботу, тощо.

Приклад

Код із бібліотеки:

```
class error_info{    .....
    error_info(const std::string& filename );
    std::string err_filename() const;
};
void do_work_on_file(const std::string& filename ){
  .....
    if(have_error_condition)
        throw error_info(filename);
  .....
}
```

Використання:

```
    try{
            ...............
        do_work_on_file("test.dat");
            ...............
    }
    catch (error_info &e)
    {
        cout << "Error, while working with file : " << e.err_filename() << endl;
    }
```

Синтаксис — 1

- Генерується виключення за допомогою ключового слова **throw**, після якого йде об'єкт-виключення.
- Щоб можна було перехопити виключення, код, який його може генерувати, слід помістити у **try**-блок:

```
try
{
    КОД, ЩО МОЖЕ ГЕНЕРУВАТИ ВИКЛЮЧЕННЯ
}
```

- Які саме виключення будуть перехоплюватися, визначається **catch**-блоками, які йдуть після try-блоку. Таких блоків може бути декілька.
- Якщо виключення не було перехоплено, програма завершить роботу викликом ф-ції terminate() (яка, по замовчуванню, викличе abort(), яка, у свою чергу, припиня виконання зразу — без виклику деструкторів, функцій, встановлених atexit()/at_quick_exit(), і т.д, але її можна замінити за допомогою set_terminate).

Синтаксис — 2

- Кожен catch-блок вказує, виключення якого типу він оброблятиме:

```
catch(exception_type_1& exc1){
    АНАЛІЗ, КОРИСТУЮЧИСЬ ВМІСТОМ exc1
}
catch(exception_type_2& exc2){
    АНАЛІЗ, КОРИСТУЮЧИСЬ ВМІСТОМ exc2
}
 ...   і тд ..   ...
```

- Перехопити всі (тобто, будь-якого типу) виключення можна за допомогою **`catch(...)`**. Якщо такий catch-блок є, він повинен бути останнім:

```
try{
    КОД, ЩО МОЖЕ ГЕНЕРУВАТИ ВИКЛЮЧЕННЯ
}
catch (...){
    cout << "Unknown error!" << endl;
}
```

- **Увага! Обробка блоків catch відбувається в порядку їх написання!**

Синтаксис — 3

- Строго кажучи, catch(A e) перехоплює наступні виключення:
  - Якщо виключення є типу A.
  - Якщо A є відкритим і однозначним[3] базовим класом виключення.
  - Якщо A – вказівник або посилання на тип, для якого виконується один із попередніх пунктів.
- Тому рекомендується **генерувати виключення по значенню, а перехоплювати — за посиланням.**
  - Передача в ролі виключення самого об'єкту (на противагу посиланням, вказівникам) різко спрощує керування його ресурсами.
  - Перехоплення за посиланням запобігає зайвим копіюванням виключення та **зрізці**, якщо перехоплюється потомок у ролі класу-предка.
  - Не-константне посилання дає можливість змінювати вмісти об'єкту виключення, наприклад — доповнювати інформацію про проблему.
- Не може бути більше одного активного виключення (така ситуація виникає, наприклад, коли деструктор, викликаний під час розкручування стеку сам кидає виключення). Якщо таке стається — програма завершується із викликом terminate(). (Забігаючи наперед, той же ефект дасть виклик "`throw;`" для повтору виключення, якщо немає що повторно кидати).
- Увага, з моменту входу в блок catch, виключення вважається обробленим, виклик throw породжує нове виключення.

[3] Згадуємо про множинне успадкування.

Синтаксис — 4

- Якщо catch-блок не може повністю опрацювати помилку, він може ініціювати повторну генерацію того ж виключення, яке перехопив. Робиться це за допомогою ключового слова throw:

```
  .....................
catch(SomeExType &e){
      .....
      if (can_manage_error)
      {      .....  }
      else{
           throw;
      }
}
```

- Слід використовувати "`throw;`" замість "`throw e;`" — в процесі генерації виключення, воно може копіюватися, не зважаючи на те, що конкретному коду воно передається за посиланням. Тоді, якщо реальний об'єкт є нащадком SomeExType, продовжить розповсюджуватися виключення вихідного типу, а "`throw e;`" зріже його до SomeExType.
- З іншого боку, якщо є потреба — можна "поглинути" вихідне виключення, піймане тут, та згенерувати інше.

Підтримка виключень

- Хоча виключення можна реалізувати без зайвих витрат пам'яті та часу виконання, іноді це складно[4].
- Тому, виключення можуть бути одним із тих нечисельних засобів C++, що сповільнити вашу програму, навіть якщо ви ними не користуєтеся[5].
- Сильно обмежені середовища, наприклад – вбудовувані (embedded) системи іноді не підтримують використання виключень.
- Тому в багатьох компіляторах передбачена можливість вмикати/вимикати підтримку виключень.

[4] Подробиці – див. далі.

[5] На сучасних реалізація – зазвичай, не буде, тому це не дуже серйозне застереження.

Як не треба

- Виключення є інструментами обробки помилок.
- В жодному разі не використовуйте їх як такий собі goto!
  - Причин для цього багато – від звичних аргументів проти goto до сповільнення виконання коду.
- Помилка – ситуація, яку неможливо виправити в коді локально. Якщо є можливість вирішити проблему локально, скористайтеся нею — виключення для нелокальних!
- Виключення дозволяють обробляти помилки структурно – скористайтеся цією можливістю.
- Тип виключення визначає різновид помилки, вміст відповідного об'єкту – подробиці.
  - Тому, хоча, синтаксично, в ролі об'єктів для виключення можуть служити і змінні фундаментальних типів, ніколи так їх не використовуйте!
  - Якщо перекласти "throw 5" на звичну мову, фраза "в мене сталася помилка int" звучить більш ніж дивно.

Ієрархії виключень

- Часто зручно оголосити цілу ієрархію класів-виключень[6]. Наприклад, базовий клас — загальна помилка у якомусь модулі, його потомки – конкретизовані варіанти:

```
class PrinterError {
        .............
    virtual std :: string get_error_text();
    virtual int get_error_code();
};

class UserCorrectableError : public PrinterError { ..... };
class HardwareError: public PrinterError { ..... };

class NoPaper: public UserCorrectableError{ ..... };
class PaperJammed: public UserCorrectableError{ ..... };
class CartridgeEmpty: public UserCorrectableError { ..... };

class CartridgeDamadged: public HardwareError {.....};
class EngineDamadged: public HardwareError {.....};
```

[6] Див., також, далі про стандартні виключення.

Тоді, наступник код перехоплюватиме всі помилки принтера:

```
try{
      ....
}
catch( PrinterError  &e){
  .......
}
```

а такий лише помилки, пов'язані із поломкою картриджа:

```
try{
      ....
}
catch(CartridgeDamadged &e){
  .......
}
```

Гарантії щодо виключень

- Програми повинні розроблятися так, щоб вони були коректними по відношенню до виключень. Стандартна бібліотека C++ пропонує кілька категорій такої коректності:
- Базова гарантія: не зважаючи на виникнення виключення, основні інваріанти всіх об'єктів залишаються дотриманими, об'єкти перебувають в коректному, самоузгодженому стані (зокрема, для них можна викликати деструктор), хоча конкретний вміст їх може бути непередбачуваним. Жодного витікання ресурсів не відбувається.
- Строга гарантія: на додачу до вимог базової гарантії, не зважаючи на виключення, операція або проходить успішно, або не має ніяких видимих ефектів.
- Гарантія відсутності виключень — найбільш строга, але можлива лише для деяких функцій (типовий приклад — swap).
- **УВАГА!** Щоб можна було давати хоч якісь гарантії щодо виключень, деструктори ніколи не повинні випускати їх за межі свого тіла! Взагалі, деструктори, що кидають виключення, породжують багато проблем. Тому, настійливо рекомендується, щоб деструктори НІКОЛИ не могли генерувати виключення.

Друга глава "Решение сложных задач на C++" Саттера якраз присвячена безпеці щодо виключень — обов'язково подивіться!. Коротко її підсумовуючи:

- Всі дії, які можуть бути небезпечними щодо виключень, слід винести в окрему функцію, щоб не вирішувати проблеми із гарантіями кожного разу наново. Решта дій виконувати, викликаючи цю функцію та функції із гарантією відсутності виключень — тієї ж swap.
- Функції, які роблять декілька змін до об'єкту, важко зробити такими, що мають строгу гарантію. Краще розділити їх на декілька (приклад — void pop_back(); і T back(); шаблону std::vector<T, ...>).
- Забезпечення строгої гарантії включає турботу про те, щоб "офіційні" вказівники, ітератори і т.д., на елементи об'єкта (зазвичай це стосується контейнерів) теж не змінювалися.
- Повторимося, деструктори не повинні кидати виключення!
- Створення безпечного щодо виключень коду сильно впливає на його проектування — про цю безпеку слід думати ще на стадії проектування!

- Аналіз гарантій безпеки щодо виключень, за участі move-операцій, для вектора: `https://quuxplusone.github.io/blog/2022/08/26/vector-pessimization/` та `https://quuxplusone.github.io/blog/2022/09/30/vector-pessimization-pick-two/`.

Специфікація виключень – викинуто в C++17

- Існує можливість вказати, які саме виключення може генерувати функція:

```
void fn(int) throw(error_info, std::exception);
void do_safe() throw();
```

- Якщо функція кине виключення, якого немає в списку, програма завершить свою роботу викликом terminate().
- throw() означає обіцянку не кидати виключення взагалі (згадаймо ту ж swap).
- Практика показала, що специфікація виключень практично не дає переваг, створює проблеми і часто приводить до повільного коду, тому не рекомендується її використовувати — цей засіб визнано застарілим!
- Єдиний виняток — гарантія відсутності виключень, throw().
- В C++11 додано специфікатор noexcept, який є статичним (часу компіляції) аналогом throw():

```
void do_safe() noexcept;
```

- В C++17 noexcept став частиною типу функції.
- Із C++17 викинуто динамічну специфікацію виключень, застарілу з часів C++11.

noexcept — C++11

- Крім специфікатора noexcept:

  void do_safe() noexcept;

- можна робити функцію noexcept лише за певних умов. Наприклад, якщо аргумент є POD:

  template<**typename** T>
  int analize (T&) noexcept(is_pod<T>);

  Відповідне рішення, звичайно, приймається під час компіляції! Просто noexcept еквівалентне noexcept(true)
- За допомогою **оператора** noexcept(), (не плутати із специфікатором noexcept), який повертає true, коли компілятор знає, що аргумент не може кидати виключення, false в іншому випадку, можна створювати і більш складні конструкції:

  template<**typename** T>
  int f(**const** T&) ????? ;
  template<**typename** T>
  int analize (T&) noexcept(noexcept(f(T)));

  Тоді analize() буде noexcept для типу T, лише якщо компілятор знає, що такою є функція f(T).
- Оператор noexcept() не обчислює переданий йому вираз.
- Крім того, noexcept() — не магія, він лише дивиться чи всі виклики в аргументі є noexcept, не занурюючись глибше, до реалізації цих операцій.

Рекомендації по використанню noexcept

Стандартна бібліотека користується наступними рекомендаціями:

- Кожна функція, яка не кидає виключень і не має невизначеної поведінки, повинна бути помічена як noexcept.
- Якщо для swap(), конструктора переміщення чи присвоєння із переміщенням, із використанням оператора noexcept() може бути перевірено, чи вони не кидають виключення, слід цим скористатися.
- Деструктори не повинні кидати виключень. В C++11 вони неявно помічені як noexcept.
- Функції, які мають бути сумісні із кодом на C, варто робити noexcept.
- В інших випадках не варто зловживати noexcept.
  - Зокрема, якщо функція може мати невизначену поведінку, коли не виконуються вимоги до її аргументів, відсутність noexcept дає можливість runtime-коду кинути відповідне (хай і опціональне) виключення.
- `https://devblogs.microsoft.com/oldnewthing/20220408-00/?p=106438` – "Please repeat yourself: The noexcept(noexcept(...)) idiom".

Посилання

- `https://devblogs.microsoft.com/oldnewthing/20220408-00/?p=106438` – "Please repeat yourself: The noexcept(noexcept(...)) idiom".
- `https://quuxplusone.github.io/blog/2024/08/16/libstdcxx-noexcept-hash/` – "'noexcept' affects libstdc++'s 'unordered_set'", есей на тему, як noexcept може впливати на конкретні реалізації. Наприклад, стандартна бібліотека GCC за наявності noexcept хеш-функції надає перевагу не кешувати хеші, а перераховувати їх наново.

Взаємодія із багатопоточністю — C++11

- Виключення не повинні покидати потік, інакше програму буде завершено викликом terminate().
- Для передачі виключення між потоками, можна використати функцію current_exception() із стандартної бібліотеки, яка повертає спеціалізований розумний вказівник на поточне виключення (яке потім можна повторно згенерувати std::rethrow_exception()).
  - Детальніше див. `http://www.cplusplus.com/reference/exception/current_exception/` і посилання там.
- Механізм, пов'язаний із current_exception() – дуже низькорівневий, тому, за потреби передавати виключення з потоків, пара future-promise може бути кориснішою.

Стандартні виключення

Стандартна бібліотека має свою ієрархію класів для повідомлення про помилки у її коді.

- Оголошені вони в файлі `<stdexcept>`:

  #include <stdexcept>

- Всі є потомками `std::exception`, оголошеного в `<exception>`:

  #include <exception>

- std::exception має віртуальний метод `what()`, який повертає стрічку із описом проблеми.
- Класи-потомки перевизначають її, щоб повертати опис помилки, яка їх породила.
- **УВАГА!** Конкретний вміст цієї стрічки залежить від компілятора, його версії і т.д.

Деякі класи виключенння стандартної бібліотеки:

- `bad_alloc` — помилка виділення пам'яті.
- `bad_cast` — помилка приведення типів, зокрема, від `dynamic_cast`.
- `logic_error` — логічна помилка в програмі. Свідчить про помилку в самій програмі.
- `runtime_error` — помилка часу виконання. Свідчить про погані вхідні дані.
- `ios_base::failure` — базовий клас помилок роботи з файлами[7].
- `bad_optional_access` — звертання до значення об'єкту, що не містить значення, (C++17).
- `bad_variant_access` — звертання до невірної альтернативи або до об'єкту у стані `valueless_by_exception`, (C++17).
- `bad_weak_ptr` — спроба скористатися об'єктом, що вже видалено (C++11).
- `bad_function_call` — викликано std::function без функції (C++11).
- `bad_exception` — езотерична річ, див. `https://en.cppreference.com/w/cpp/error/bad_exception`.

[7] Починаючи з C++11 – нащадок `system_error`, що є нащадком `runtime_error`.

Нащадки `bad_alloc`:

- `bad_array_new_length` — від'ємне або завелике значення для new[], або забагато ініціалізаторів (C++11).

Нащадки `logic_error`:

- `domain_error` — помилка області визначення функцій (наприклад, від'ємне число в ролі аргументу sqrt()).
- `invalid_argument` — невірний аргумент.
- `length_error` — помилка вказання довжини. Наприклад, спроба створити завеликий контейнер.
- `out_of_range` — вихід за межі діапазону.
- `future_error` — може кидатися за різних видів помилок future-promise, подробиці у відповідній лекції (C++11).

Нащадки `bad_cast`:

- `bad_any_cast` — кидається std::any_cast (C++17).

Нащадки `runtime_error`:

- `overflow_error` — арифметичне переповнення.
- `range_error` — вихід за межі діапазону (для "зовнішніх" даних програми).
- `underflow_error` — втрата точності в арифметичних числах з рухомою комою.
- `regex_error` — помилки відповідної стандартної бібліотеки (C++11).
- `system_error` — виключення, що кидається різними частинами бібліотеки, якщо виключення має пов'язаний код помилки std::error_code (C++11).
  - `ios_base::failure` — переміщено сюди в C++11.
  - `filesystem::filesystem_error` — помилки відповідної бібліотеки (C++17).
- `chrono::nonexistent_local_time` та `chrono::ambiguous_local_time` – дещо езотеричні помилки бібліотеки chrono при перетворенні локального часу в системний (C++20).
- `format_error` — кидається функціями відповідної бібліотеки (C++20).

Приклад:

```
int* laaaaarge_data;
try
{
    laaaaarge_data = new int[INT_MAX/4-1];
}
catch (std :: exception  &e)
{
    cout << "An exception occurred: " << e.what() << endl;
}
```

```
ifstream no_such_file;
// Дозволимо виключення для цього потоку:
no_such_file.exceptions( ios :: badbit | ios :: failbit );
try
{
    no_such_file.open("absent.nodat");
}
catch (std :: exception &e)
{
    cout << "An exception occurred: " << e.what() << endl;
}
catch (...)
{
    cout << "Unknown error!" << endl;
}
```

Дасть:

```
An exception occurred: ios_base::failbit set
```

Тоді як:

```
try
{
    throw 13; // Ніколи так не робіть!
}
catch (std :: exception &e)
{
    cout << "An exception occurred: " << e.what() << endl;
}
catch (...)
{
    cout << "Unknown error!" << endl;
}
```

Дасть:

```
Unknown error!
```

```
int some_fn(const int x)
{
    if (x<0)
        throw range_error("Negative values prohibited!");
    return -x;
}

    try
    {
        some_fn(-5);
    }
    catch (std::exception &e)
    {
        cout << "An exception occurred: " << e.what() << endl;
    }
```

Дасть:

```
An exception occurred: Negative values prohibited!
```

Ціна виключень

- Існує два підходи до реалізації виключень.
  - Перший: динамічно оновлювати список задач (наприклад, викликів деструкторів), які потрібно виконати, коли сталося виключення.
  - Другий: використовувати додаткову інформацію, збережену у виконавчому файлі, для визначення того, що робити. У цій ролі використовується відладочна інформація, яка пов'язує об'єктний код із текстом програми.
- Перший підхід сповільнює весь код, де можуть бути виключення. Тому сучасні реалізації ним не користуються.
- Другий сам по собі не накладає додаткових витрат на виконання, **у випадку, коли виключень немає**.
- Однак, обробка виключень при тому повільна!
- Крім того, це забороняє багато оптимізацій. Наприклад, компілятор мусить зберігати стан всіх видимих обробникам виключень об'єктів у пам'ять (так званий spill)– тоді як в нормі багато що могло залишатися в регістрах. Також, він більш обмежений у перестановках операцій – які б дозволили швидше виконувати код сучасним процесорам.

Ціна виключень – посилання

- https://habr.com/ru/company/jugru/blog/494986/ – приклад перепон оптимізатору, спричинених можливістю виключень, навіть якщо їх не відбувається.
- Інша стаття на ту ж тему: https://devblogs.microsoft.com/oldnewthing/20220228-00/?p=106296, "Zero-cost exceptions aren't actually zero cost".
- https://devblogs.microsoft.com/oldnewthing/20220418-00/?p=106489 – "The x86 architecture is the weirdo, part 2", стосовно особливостей обробки виключень.
- http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2544r0.html – "C++ exceptions are becoming more and more problematic", про ціну на багатоядерних процесорах (оскільки розкрутка зараз потребує доступу до глобальних структур) та способи вирішення. Російський переклад: https://habr.com/ru/company/skillfactory/blog/654723/.
- https://www.sandordargo.com/blog/2025/05/07/cpp26-constexpr-exceptions – "C++26: constexpr exceptions".

Function try-blocks

- Все тіло функції може бути try-блоком:

```
Drib div(Drib x, Drib y)
try
{
    return x / y;
}
catch (...)
{
    return Drib(0);
}
```

- Основне призначення такої конструкції — перехоплення виключень в конструкторах під час конструювання полів, див. наступний слайд.
- Можна також використовувати і в деструкторах.

Try-blocks конструкторів

```
class X{    ..............
    A a;
    B b;

    X():
    try
        a(1),
        b(3,3)
    {
            .............
    }
    catch(error_info & e){
            ............
    }
};
```

- На відміну від try-блоків функцій, try-блок конструктора не може "відмінити" виключення — він може хіба якось повідомити про нього, залогувати якусь інформацію і т.д.
  - ◇ Використовувати return в ньому не можна, а досягнення кінця тіла catch-блоку дасть результат, еквівалентний `throw;`. Все рівно, об'єкт, конструювання полів якого провалилося, не може бути сконструйованим.
- Якщо об'єкт має якісь опціональні поля, працювати без яких може (хай — в урізаному варіанті), слід надати доступ до них за вказівником і конструювати в тілі конструктора, де виключення можна успішно перехоплювати.
  - ◇ Часто це – погано спроектований клас.
- Оскільки для конструкторів try-блоки функцій вирішують лише дуже обмежену задачу, а для функцій та деструкторів взагалі не несуть нічого нового, цей засіб не є особливо корисним.
- Способів перехопити виключення глобальних змінних та змінних рівня просторів імен не існує.
- Див. також другу главу "Решение сложных задач на C++" Саттера.

std::nested_exception (C++11)

- std::nested_exception – клас-mixin, що дозволяє вкладати одні виключення в інші.
- Приклад:

```
// prints the explanatory string of an exception. If the exception is nested,
// recurses to print the explanatory of the exception it holds
void print_exception(const std::exception& e, int level = 0){
    std::cerr << std::string(level, ' ') << "exception: " << e.what() << '\n';
    try {
        std::rethrow_if_nested(e);
    } catch(const std::exception& nestedException) {
        print_exception(nestedException, level+1);
    } catch (...) {}
}

// sample function that catches an exception and wraps it in a nested exception
void open_file(const std::string& s){
    try {
        std::ifstream file(s);
        file.exceptions(std::ios_base::failbit);
    } catch (...) {
        std::throw_with_nested( std::runtime_error("Couldn't open " + s) );
    }
}
```

- Приклад із `https://en.cppreference.com/w/cpp/error/nested_exception`.

std::exception_ptr – C++11

- std::exception_ptr – спеціальний вказівник, для збереження поточного виключення за допомогою std::current_exception:

```
std::exception_ptr e_ptr;

try {
    ....
    throw std::logic_error("Error");
    ...
} catch (...) {
    e_ptr = std::current_exception();
}
```

- Таким чином виключення може "пережити" try-catch, не опрацьованим, але і не зникнути. Далі можна зробити щось таке, навіть у іншому потоці:

```
if (e_ptr) {
    std::rethrow_exception(e_ptr);
}
```

- std::rethrow_exception – єдиний спосіб добути щось з нього, звернутися до what() абощо по іншому не вдасться.
- std::make_exception_ptr – створити такий вказівник з об'єкту-виключення, так ніби було виконано throw-catch.
- З допомогою цього інструменту можна передавати виключення між потоками та обробляти виключення у деструкторах – які дуже небажано "випускати" з них.

Пропозиція std::error

- `https://github.com/charles-salvia/std_error` – експериментальна реалізація пропозиції щодо нового підходу до обробки помилок: "Zero-overhead deterministic exceptions: Throwing values", `https://wg21.link/p0709`.
- Станом на 2026, версія завмерла на варіанті від 2019 року, в нові стандарти не потрапило. Є ідейний продовжувач: "https://wg21.link/p3166".

Посилання

- `https://quuxplusone.github.io/blog/2023/11/11/guard-nouns-or-guard-verbs/` – цікаві міркування, які, поміж іншого, зачіпають noexcept/
- Есей про обробку помилок, за допомогою кодів помилок: "C++ Enum Class and Error Codes", "part 2" та, власне, виключень: "part 3".
- "Why is my first C++ (m)allocation always 72 KB?" – особливо це шокувало, коли воно так на MCU з 4 Кб RAM спробувало...

